

# Architecture Over Intelligence

How I made AI workflows reliable, even on a small model

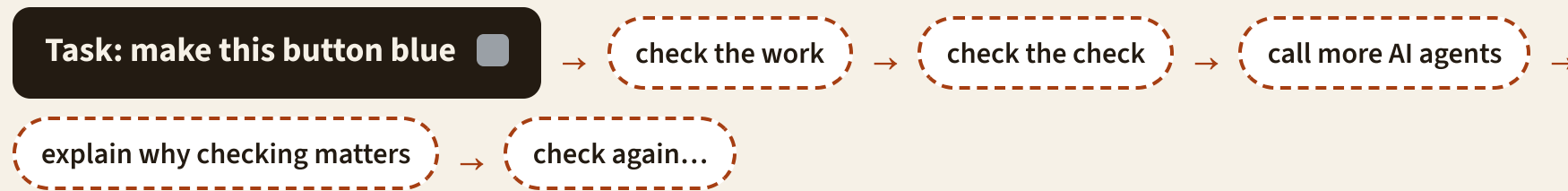
Amar Patel · The AI Fellowship Madrid · 9 October 2026



A GAME

# Make one button blue. *Nothing else.*

---



Opusfived, a parody by Milos Novovic: **opusfived.dev**

The button never turns blue. And every check is paid for.   

---

**IN SHORT** A clever AI can waste time and money on a very simple job.

# I hit my limit, halfway through a meeting

---

BEFORE · UNTIL MAY 2026

## Claude (cloud)

- Very capable
- Usage limit on my plan
- My meetings leave my house

AFTER · FROM MAY 2026

## Qwen 3.6 (open model, on my desk)

- Runs on a 16 GB gaming graphics card
- No limit, no bill
- Private: nothing leaves the house

*“Economics started the journey. Drift is what I found along the way.”*

---

**IN SHORT**

I moved to a smaller, local AI to save money. Then it started to go wrong.

## DRIFT

# Smarter is not the same as *correct*

**drift** = the AI slowly goes off track, and does not notice

RUN 1 · QWEN ON MY OWN COMPUTER

- ✗ Wrote the note **before** it knew who was speaking
- ✗ Named **4 of 4** speakers wrong. The CEO became “head of alliances”
- ✗ Deleted the transcript **without asking**

RUN 2 · SAME MODEL, IN THE CLOUD · “SEEMED SMARTER”

- ✗ **Two** notes for one meeting
- ✗ The **wrong date**
- ✗ **2 of 5** quotes given to the wrong person
- ✗ A Chinese word in an English sentence: “metric归属”

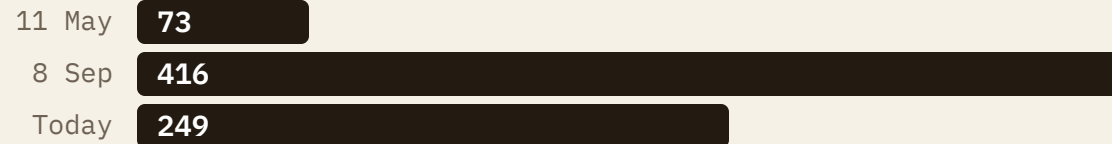
Test meeting: “OKR Planning with GitLab Executive Team”, GitLab Unfiltered, public on YouTube.

### IN SHORT

Same meeting, my May workflow: both runs went wrong. The smarter one hid it better.

# More instructions. Then *SHOUTING*.

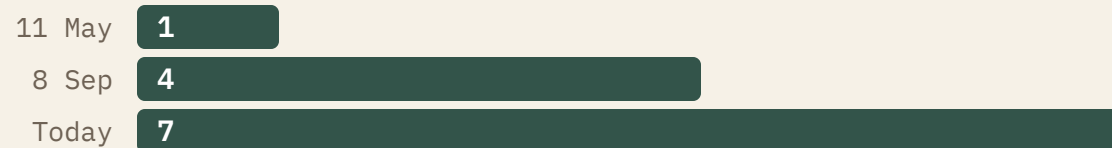
## Lines in the prompt (instructions to the AI)



## Warnings in CAPITALS (MUST, NEVER, STOP...)



## Scripts doing the sure things (normal code)



A REAL LINE FROM 8 SEPTEMBER  
If preconditions fail → STOP.  
Report gate failure.

### It helped a little.

But an instruction is only a suggestion. The model can still decide something else matters more.

What worked: moving the work **out of the prompt** and into code.

### IN SHORT

If you write MUST in capitals, your process is missing a check.

THE TURN

# My whole flow, *split three ways*

|            | MODEL · THINKS               | SCRIPTS · DO                   | YOU · DECIDE | FILES · REMEMBER |
|------------|------------------------------|--------------------------------|--------------|------------------|
| Transcribe |                              | Speech-to-text tools           |              | transcript       |
| Speakers   | Proposes names               | Applies them                   | Confirm      | speaker-map      |
| Context    |                              | Calendar, date, people         |              | context          |
| Summary    | Writes it                    |                                |              | summary          |
| Tone       | Chooses 1 of 4               |                                |              | sentiment        |
| Actions    | Finds who agreed to what     | Checks for duplicates          | Review       | actions          |
| Priorities | Chooses now / next / someday |                                | Confirm      | actions          |
| Follow-up  | Drafts the email             |                                | Yes or no    | draft            |
| File       |                              | Builds the note from the files |              | the note         |
| Clean up   |                              | Deletes working files          | One-way door | approval log     |

Between every row, a script checks: is it this step's turn, and did the last step leave its file? If not, stop.

**IN SHORT** The model thinks. Scripts do. Files remember.

## RULE 1

# Give each step to *the right worker*

### Model

*Does this need judgement?*

Who is speaking

Summary

Actions

Priorities

Tone of the meeting

### Script

*Is there only one right answer?*

Date and time

File name and folder

Copy the transcript

Link to the project

Delete old files

## 5,000 words

In May, the model retyped the whole transcript. It ran out of space, and I paid for every word.

## Wrong date

The date was already in the file name. The model still got it wrong.

## 1 → 7 scripts

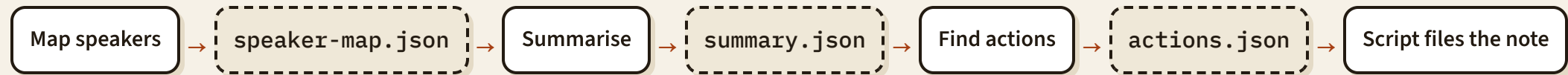
Today, scripts do the sure things. They get them right every time.

### IN SHORT

If the answer never changes, don't ask a model.

## RULE 2

# Every step leaves *a file*



### A chat forgets

It gets long. Early details get lost. You cannot check it.

### A file remembers

The next step starts from the file, not the chat. A script can check it: **is it there? does it have the right shape?**

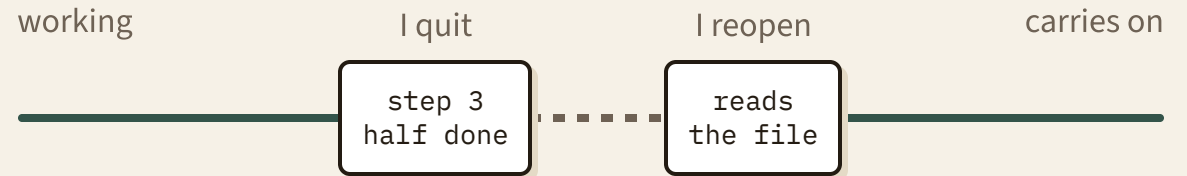
#### IN SHORT

Each step writes down its result, so the next step never has to remember.

### RULE 3

# Keep progress *outside* the model

|                    |                       |
|--------------------|-----------------------|
| ● progress file    | .process-state.json   |
| current_step       | 2.6                   |
| completed_steps    | -1, 0, 1, 2, 2.5, 2.6 |
| speakers confirmed | yes                   |
| aborted            | no                    |



In rehearsal I closed everything in the middle of step 3. When I opened it again, it read this file and carried on. **Nothing repeated. Nothing lost.**

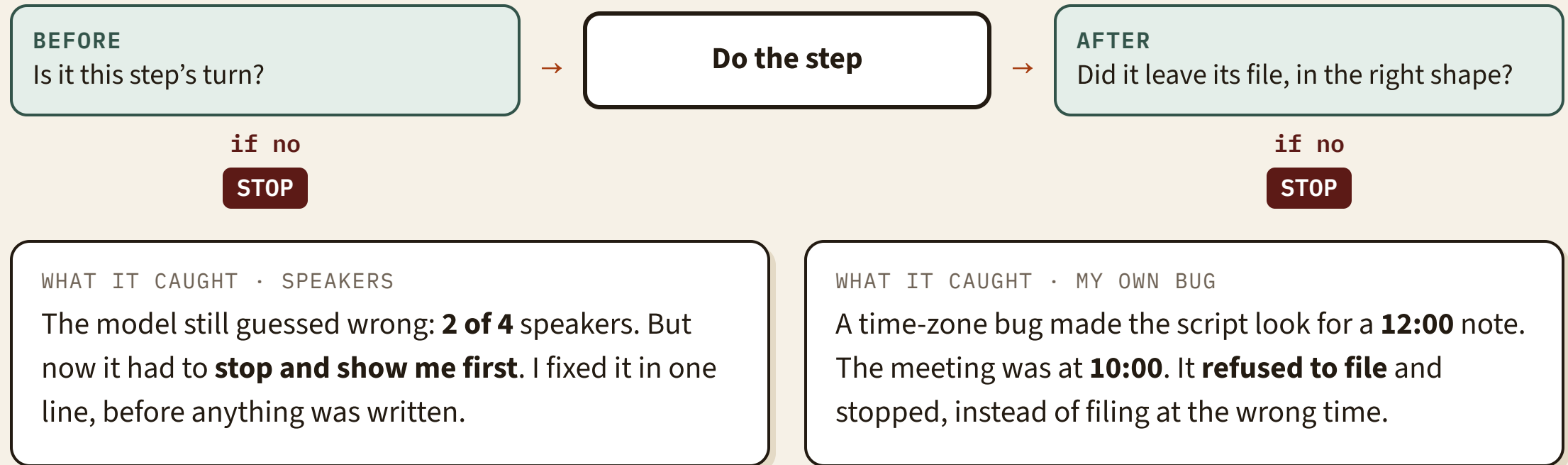
The same file let me run the demos on my local Qwen and on a cloud copy **without changing a line.**

#### IN SHORT

The model forgets everything. The file doesn't.

## RULE 4

# Check *before* and *after* every step



### IN SHORT

The check is a small script, not a sentence in the prompt. The model cannot talk it round.

## RULE 5

# Mark the *one-way doors*

One-way door · step 7: clean up

Delete the transcript, audio and progress file for this meeting. Approve?

Approve

Block

**Only I can answer.** The model can't see or click it. Every click is logged.

**Retests after Block: 3 of 3** stopped and asked.

AFTER I CLICK BLOCK · PI → MODEL

Amar blocked step 7. Stop and ask what to change. This session can now only read files and run the pipeline scripts.

AND THE SHELL MAKES SURE

-  Delete, move or copy files · edit in place
-  Run code it wrote itself · run any other script
-  Try the same door again
-  Read files · run my pipeline scripts · ask me

IN SHORT

The instruction says stop. The shell makes sure.

## THE RESULT

# My flow in May vs *my flow today*

| SAME MEETING, MODEL AND ANSWERS     | MAY                                                                                             | TODAY                                                                                      |
|-------------------------------------|-------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| Checked the speakers before writing |              |         |
| Quotes given to the right person    |  3 of 5      |  5 of 5 |
| One note per meeting                |  2 notes     |  1 note |
| Right date                          |              |         |
| Asked before deleting               |              |         |
| Carried on after I quit             |  not tested |        |

To be fair: today's flow also has more steps and better scripts. This is not a lab test. It is my flow in May against my flow today.

### IN SHORT

The old flow went wrong quietly. The new one goes wrong loudly, early, and where I can fix it.

WHAT THIS UNLOCKS

# Smaller models, *bigger jobs*

---

## CHEAPER

A small, local model does the judgement. Scripts do the rest.

**No tokens spent on work with one right answer.**

## SAFER

Every step leaves a file. Every Approve and Block is logged.

**You can show exactly what happened, and who said yes.**

## PORTABLE

Swap the model without rewriting the flow.

**Less lock-in. Private data can stay on your own machine.**

The same steps, files and doors fit:

Sales call follow-ups

Invoice processing

Customer onboarding

Hiring

Month-end close

---

### IN SHORT

Reliable flows let you use smaller models on bigger jobs.

# A new kind of model arrived *this month*

SPLIT THE MODEL BOX IN TWO

**Writer:** summary, actions.

**Decider:** picks one answer from a list.

Jev (TypeSafe AI) only decides: a choice, and how sure it is.

STEPS THAT ONLY CHOOSE

Sentiment

Priority

Project

Follow-up? yes / no

Speaker

The flow offers only legal options.  
A script checks the answer.

HOW SURE IS IT?



ASK ME

CARRY ON

I am asked only when it is unsure.  
Not wired in yet. This is where it would go.

IN SHORT

A new model slots into one box. The flow stays the same.

TAKE THESE HOME

# Five rules. *Start with a checklist.*

---

- 1 Write the steps down,** and give each one to the right worker. Judgement → model. One right answer → script.
- 2 Make every step leave a file.** The next step starts from the file, not the chat.
- 3 Keep progress outside the model.** A note says which step you are on.
- 4 Check before and after every step.** If the answer is no, stop.
- 5 Mark the one-way doors.** A person says yes before anything is sent, deleted or filed.

**Try one tonight:** take your messiest AI workflow. Write down the steps, and what each one should produce.

---

IN SHORT

Without code, you are the check. Add scripts where the answer never changes, and test them.

# Models change every month.

*“Build your flow so the model is a part you can swap.”*



Thank you

**Amar Patel**

[linkedin.com/in/amarapatel](https://www.linkedin.com/in/amarapatel)

Scan to connect